ASTAS Workflow & Usage Guide

🚀 Quick Start Guide

Installation

```bash
# 1. Clone repository
git clone https://github.com/yourusername/ASTAS.git
cd ASTAS

# 2. Create virtual environment
python -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate

# 3. Install dependencies
pip install -r requirements.txt

# 4. Download YOLOv8 model (optional - will use mock if not available)
# The system will automatically download on first run if ultralytics is installed
```

Running ASTAS

```bash
# Basic simulation (60 seconds)
python main.py

# Custom duration and FPS
python main.py --duration 120 --fps 30

# With custom config
python main.py --config my_config.yaml
```

📊 Complete Workflow Diagram

```
┌─────────────────────────────────────────────────────────────────────────┐
│                          START ASTAS SYSTEM                             │
└─────────────────────────────────────────────────────────────────────────┘
                                  │
```

```
                    ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│ INITIALIZATION PHASE                                                         │
├──────────────────────────────────────────────────────────────────────────────┤
│ 1. Load Configuration (config.yaml)                                          │
│ 2. Initialize Computer Vision (YOLOv8 Detector)                              │
│ 3. Initialize Motion Tracker (DeepSORT)                                      │
│ 4. Initialize Sensors (IMU, LiDAR, Audio)                                    │
│ 5. Initialize LLM Decision Engine                                            │
│ 6. Initialize Camera Controller (PID)                                        │
│ 7. Setup Alert System & Logging                                              │
└──────────────────────────────────────────────────────────────────────────────┘
                    │
                    ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│ MAIN PROCESSING LOOP (30 FPS)                                                │
└──────────────────────────────────────────────────────────────────────────────┘
                    │
          ┌─────────┴─────────┐
          │                   │
          ▼                   ▼
┌──────────────────────┐   ┌──────────────────────┐
│ FRAME CAPTURE        │   │ SENSOR CAPTURE       │
├──────────────────────┤   ├──────────────────────┤
│ • RGB Camera         │   │ • IMU Data           │
│ • Thermal (opt)      │   │ • LiDAR Scan         │
│ • Resolution:        │   │ • Audio Stream       │
│   1920x1080          │   │ • Timestamps         │
└──────────────────────┘   └──────────────────────┘
          │                   │
          └─────────┬─────────┘
                    │
                    ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│ STEP 1: OBJECT DETECTION (YOLOv8)                                            │
├──────────────────────────────────────────────────────────────────────────────┤
│ Input: RGB Frame (H x W x 3)                                                 │
│ Process:                                                                     │
│   - Forward pass through neural network                                      │
│   - Non-Maximum Suppression (NMS)                                            │
│   - Confidence filtering (threshold: 0.5)                                    │
│ Output:                                                                      │
│   Detections = [                                                             │
│     {class: "person", conf: 0.92, bbox: [x1,y1,x2,y2]},                      │
```

```
│    {class: "car", conf: 0.87, bbox: [x1,y1,x2,y2]},          │
│    ...                                                       │
│  ]                                                           │
│ Latency: ~30-40ms (GPU) / ~200ms (CPU)                       │
└──────────────────────────────────────────────────────────────┘
                    │
                    ▼
┌──────────────────────────────────────────────────────────────┐
│ STEP 2: MULTI-OBJECT TRACKING (DeepSORT)                     │
├──────────────────────────────────────────────────────────────┤
│ Input: Current Detections + Previous Tracks                  │
│ Process:                                                     │
│   - Predict track positions (Kalman filter)                  │
│   - Match detections to tracks (IoU matching)                │
│   - Update matched tracks                                    │
│   - Create new tracks for unmatched detections               │
│   - Delete old tracks (max_age exceeded)                     │
│ Output:                                                      │
│   Tracks = [                                                 │
│    {id: 1, class: "person", bbox: [...], trajectory: [...],  │
│     velocity: (vx, vy), age: 45},                            │
│    ...                                                       │
│  ]                                                           │
│ Latency: ~5-8ms                                              │
└──────────────────────────────────────────────────────────────┘
                    │
                    ▼
┌──────────────────────────────────────────────────────────────┐
│ STEP 3: BEHAVIOR ANALYSIS                                    │
├──────────────────────────────────────────────────────────────┤
│ For each track:                                              │
│   - Calculate average speed from trajectory                  │
│   - Detect loitering (speed < 5 pixels/frame for >60 frames) │
│   - Count direction changes (angle diff > 45°)               │
│   - Classify motion type (rapid/slow/stationary)             │
│ Output:                                                      │
│   Behaviors = [                                              │
│    {track_id: 1, loitering: True, avg_speed: 2.3,            │
│     direction_changes: 3, motion: "slow"},                   │
│    ...                                                       │
│  ]                                                           │
│ Latency: ~2ms                                                │
└──────────────────────────────────────────────────────────────┘
                    │
```

```
                    ▼
┌──────────────────────────────────────────────────────────────────────┐
│ STEP 4: SENSOR PROCESSING (PARALLEL)                                 │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│ IMU Processing:                                                      │
│   - Apply Kalman filter to accel/gyro data                           │
│   - Estimate orientation (roll, pitch, yaw)                          │
│   - Detect vibrations (magnitude > threshold)                        │
│   - Classify motion (stationary/slow/moderate/rapid)                 │
│   Output: {vibration: False, motion_type: "slow", ...}               │
│                                                                      │
│ LiDAR Processing:                                                    │
│   - Downsample point cloud (voxel grid)                              │
│   - Remove ground plane (RANSAC)                                     │
│   - Cluster objects (Euclidean clustering)                           │
│   - Classify objects (person/vehicle/unknown)                        │
│   Output: [{type: "person", distance: 15.2m, ...}, ...]              │
│                                                                      │
│ Audio Processing:                                                    │
│   - Compute FFT spectrum                                             │
│   - Extract features (spectral centroid, rolloff, ...)               │
│   - Match event signatures                                           │
│   Output: [{event: "footsteps", conf: 0.75, ...}, ...]               │
│                                                                      │
│ Latency: ~10-15ms total                                              │
└──────────────────────────────────────────────────────────────────────┘
                    │
                    ▼
┌──────────────────────────────────────────────────────────────────────┐
│ STEP 5: CONTEXT BUILDING                                             │
├──────────────────────────────────────────────────────────────────────┤
│ Aggregate all data into unified context:                             │
│                                                                      │
│ context = {                                                          │
│     # From CV                                                        │
│     "num_detections": 2,                                             │
│     "detections": ["person", "car"],                                 │
│     "primary_object": "person",                                      │
│     "loitering": True,                                               │
│     "rapid_movement": False,                                         │
│     "direction_changes": 3,                                          │
│                                                                      │
│     # From Environment                                               │
```

```
│    "zone": "restricted",  # From geo-fencing                    │
│    "time_of_day": "night",  # From system clock                  │
│    "restricted_area": True,                                      │
│                                                                  │
│    # From Sensors                                                │
│    "audio_events": ["footsteps"],                                │
│    "vibration": False,                                           │
│    "lidar_objects": 2,                                           │
│                                                                  │
│    # From History                                                │
│    "previous_alerts": 1,                                         │
│    "unusual_pattern": False                                      │
│ }                                                                │
│                                                                  │
│ Latency: <1ms                                                    │
└──────────────────────────────────────────────────────────────────┘
                    │
                    ▼
┌──────────────────────────────────────────────────────────────────┐
│ STEP 6: LLM THREAT ASSESSMENT                                    │
├──────────────────────────────────────────────────────────────────┤
│ Option A: LLM-Based (if model loaded)                            │
│   - Format context as natural language prompt                    │
│   - Query fine-tuned LLM (LLaMA/Mistral + LoRA)                  │
│   - Parse JSON response                                          │
│                                                                  │
│ Option B: Rule-Based (fallback)                                  │
│   threat_score = 0.0                                             │
│   if restricted_area: score += 0.3                               │
│   if time == "night": score += 0.2                               │
│   if loitering: score += 0.25                                    │
│   if "gunshot" in audio_events: score = 1.0                      │
│   if vibration: score += 0.1                                     │
│   if unusual_pattern: score += 0.15                              │
│   ... (see detailed rules in decision_engine.py)                 │
│                                                                  │
│   # Classify based on score                                      │
│   if score < 0.4: level = "LOW"                                  │
│   elif score < 0.65: level = "MEDIUM"                            │
│   elif score < 0.85: level = "HIGH"                              │
│   else: level = "CRITICAL"                                       │
│                                                                  │
│ Output:                                                          │
│   assessment = {                                                 │
```

```
│    "threat_level": "HIGH",                                  │
│    "threat_score": 0.78,                                    │
│    "confidence": 0.85,                                      │
│    "reasoning": "Person loitering in restricted zone at night", │
│    "recommended_actions": [                                 │
│        "Alert operator",                                    │
│        "Track continuously",                                │
│        "Prepare response"                                   │
│    ]                                                        │
│  }                                                          │
│                                                             │
│ Latency: 150-300ms (LLM) / 1-2ms (rules)                    │
└─────────────────────────────────────────────────────────────┘
                    │
                    ▼
┌─────────────────────────────────────────────────────────────┐
│ STEP 7: DECISION & CONTROL                                  │
├─────────────────────────────────────────────────────────────┤
│ Based on threat level:                                      │
│                                                             │
│ IF threat_level == "LOW":                                   │
│   - Continue normal monitoring                              │
│   - No alerts generated                                     │
│                                                             │
│ IF threat_level == "MEDIUM":                                │
│   - Increase monitoring frequency                           │
│   - Log event                                               │
│                                                             │
│ IF threat_level == "HIGH":                                  │
│   - Generate operator alert                                 │
│   - Initiate camera tracking                                │
│   - Log event with details                                  │
│   - Increase sensor polling                                 │
│                                                             │
│ IF threat_level == "CRITICAL":                              │
│   - IMMEDIATE ALERT (audio + visual)                        │
│   - Contact security personnel                              │
│   - Lock down area (if applicable)                          │
│   - Record video evidence                                   │
│   - Track all objects in scene                              │
│                                                             │
│ Camera Control (if tracking enabled):                       │
│   1. Select primary target (highest threat)                 │
│   2. Calculate pan/tilt error                               │
```

```
│  3. PID controller computes speeds                    │
│  4. Apply smoothing and rate limits                   │
│  5. Send commands to PTZ camera                       │
│                                                       │
│ Latency: ~5ms                                         │
└───────────────────────────────────────────────────────┘
                  │
                  ▼
┌───────────────────────────────────────────────────────┐
│ STEP 8: OUTPUT & LOGGING                              │
├───────────────────────────────────────────────────────┤
│ 1. Visual Output:                                     │
│    - Annotated frame with bounding boxes              │
│    - Track IDs and trajectories                       │
│    - Threat level indicator                           │
│    - Zone overlays                                    │
│                                                       │
│ 2. Alert Output:                                      │
│    - Console messages                                 │
│    - Email notifications (HIGH/CRITICAL)              │
│    - SMS alerts (CRITICAL)                            │
│    - Sound alarms                                     │
│                                                       │
│ 3. Data Logging:                                      │
│    - Event database (SQLite/PostgreSQL)               │
│    - Video recordings (if enabled)                    │
│    - Sensor logs                                      │
│    - Performance metrics                              │
│                                                       │
│ 4. Performance Display:                               │
│    - FPS counter                                      │
│    - Detection count                                  │
│    - Active tracks                                    │
│    - System status                                    │
└───────────────────────────────────────────────────────┘
                  │
                  ▼
┌───────────────────────────────────────────────────────┐
│ FRAME COMPLETE - REPEAT FOR NEXT FRAME                │
└───────────────────────────────────────────────────────┘
      │
      └──────────> Back to Frame Capture (30 FPS = 33ms cycle)
```

🎯 Role of Each Module in the Workflow

1. main.py - System Orchestrator

Role: Central coordinator that manages the entire pipeline **Responsibilities:**

- Initialize all modules with config
- Manage main processing loop
- Coordinate data flow between modules
- Handle errors and exceptions
- Provide CLI interface

2. cv/object_detection.py - Primary Perception

Role: First stage of visual understanding **Responsibilities:**

- Detect all objects in frame
- Classify object types
- Provide bounding boxes and confidence
- Filter by surveillance-relevant classes

3. cv/motion_tracking.py - Temporal Understanding

Role: Track objects across time **Responsibilities:**

- Maintain object identities
- Record movement trajectories
- Calculate velocities
- Detect behavioral patterns

4. cv/thermal_analysis.py - Enhanced Vision

Role: Night vision and heat detection **Responsibilities:**

- Process infrared imagery
- Detect heat signatures
- Classify by temperature
- Fuse with RGB for enhanced view

5. sensors/imu_processing.py - Motion Sensing

Role: Detect platform movement and vibrations **Responsibilities:**

- Filter noisy IMU data
- Estimate orientation
- Detect anomalous vibrations
- Classify motion patterns

6. sensors/lidar_processing.py - 3D Perception

Role: Accurate distance and 3D mapping **Responsibilities:**

- Process point clouds
- Detect 3D objects
- Measure accurate distances
- Generate occupancy maps

7. sensors/audio_signal_analysis.py - Acoustic Awareness

Role: Detect audio events **Responsibilities:**

- Analyze frequency spectrum
- Match acoustic signatures
- Estimate sound direction
- Detect specific events (gunshots, alarms)

8. llm/inference/decision_engine.py - Intelligence Layer

Role: High-level reasoning and decision making **Responsibilities:**

- Aggregate multi-modal data
- Assess threat levels
- Provide reasoning explanations
- Recommend actions

9. control_system/pid_controller.py - Response System

Role: Smooth and accurate control **Responsibilities:**

- Track moving targets with camera
- Plan patrol routes

- Apply acceleration limits
- Ensure smooth movements

10. simulation/env_sim.py - Testing Environment

Role: Generate realistic test scenarios **Responsibilities:**

- Simulate multiple agents
- Create diverse behaviors
- Test edge cases
- Validate system performance

📈 Performance Optimization Tips

For Real-Time Performance:

1. **Use GPU for Detection**

```python
detector = ObjectDetector(device='cuda')
```

2. **Reduce Detection Frequency**

```python
if frame_count % 3 == 0:  # Detect every 3rd frame
    detections = detector.detect(frame)
```

3. **Lower Camera Resolution**

```yaml
camera:
  resolution: [1280, 720]  # Instead of 1920x1080
```

4. **Use Quantized LLM**

```yaml
```

```yaml
  llm:
    use_quantization: true
    quantization_bits: 4
```

5. **Limit Sensor Processing**

```yaml
  sensors:
    imu:
      sampling_rate: 50  # Lower from 100Hz
```

🧪 Testing Scenarios

Scenario 1: Normal Operation

```python
# Simulates routine monitoring with no threats
python main.py --duration 30 --fps 30
```

Scenario 2: Perimeter Breach

```python
# Use simulation with perimeter breach behavior
# Edit config to enable border_patrol scenario
```

Scenario 3: High Activity

```python
# Multiple agents with various behaviors
# Modify env_sim.py to spawn more agents
```

📝 Configuration Guide

Key Configuration Parameters:

```yaml
detection:
  confidence_threshold: 0.5  # Lower = more detections
  iou_threshold: 0.45        # Lower = fewer duplicate detections

tracking:
  max_age: 30                # Frames to keep track without detection
  min_hits: 3                # Frames needed to confirm track

threat_assessment:
  zones:
    red:
      threat_multiplier: 2.0  # Doubles threat score in red zones
```

End of Workflow Guide